



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

ACEVEDO FUENTES ARNOL FABIAN

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos

Actividad 1: Caza de Errores (60 minutos) 🕵️



- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

Este es la parte de código con error:

```
# Inverse of the matrix using the formula (1/determinant) * adjoint matrix
```

```
Inverse_matrix = array(cofactor_matrix)
```

```
For i in range(3):
```

```
    For j in range(3):
```



```
Inverse_matrix[i][j] /= d(determinant)
```

Y está es el corregido:

```
# Inverse of the matrix using the formula (1/determinant) * adjoint matrix
```

```
Inverse_matrix = array(adjoint_matrix)
```

```
For i in range(3):
```

```
    For j in range(3):
```

```
        Inverse_matrix[i][j] /= d(determinant)
```

El código original cometía un error lógico, Calculaba correctamente `adjoint_matrix`

Pero después no la utilizaba. En vez de eso, dividía `cofactor_matrix`

Por eso la inversa de matrices 3x3 podía salir incorrecta

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.



Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔒

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️



Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").



5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Karen Yineida Fernández Páez

Juan Pablo Russo Jauregui

Nathan Camilo Rodríguez Romero

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "inverse_of_matrix.py" que se encuentra dentro de la carpeta "matriz" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

SOLUCION

Código corregido:

```
from future import annotations
from decimal import Decimal
from numpy import array
```




```
def inverse_of_matrix(matrix: list[list[float]]) -> list[list[float]]:
    """
    A matrix multiplied with its inverse gives the identity matrix.
    This function finds the inverse of a 2x2 and 3x3 matrix.
    If the determinant of a matrix is 0, its inverse does not exist.

    Sources for fixing inaccurate float arithmetic:
    https://stackoverflow.com/questions/6563058/how-do-i-use-accurate-float-
    arithmetic-in-python
    https://docs.python.org/3/library/decimal.html

    Doctests for 2x2
    >>> inverse_of_matrix([[2, 5], [2, 0]])
    [[0.0, 0.5], [0.2, -0.2]]
    >>> inverse_of_matrix([[2.5, 5], [1, 2]])
    Traceback (most recent call last):
    ...
    ValueError: This matrix has no inverse.
    >>> inverse_of_matrix([[12, -16], [-9, 0]])
    [[0.0, -0.1111111111111111], [-0.0625, -0.08333333333333333]]
    >>> inverse_of_matrix([[12, 3], [16, 8]])
    [[0.16666666666666666, -0.0625], [-0.3333333333333333, 0.25]]
    >>> inverse_of_matrix([[10, 5], [3, 2.5]])
    [[0.25, -0.5], [-0.3, 1.0]]

    Doctests for 3x3
    >>> inverse_of_matrix([[2, 5, 7], [2, 0, 1], [1, 2, 3]])
    [[2.0, 5.0, -4.0], [1.0, 1.0, -1.0], [-5.0, -12.0, 10.0]]
    >>> inverse_of_matrix([[1, 2, 2], [1, 2, 2], [3, 2, -1]])
    Traceback (most recent call last):
    ...
    ValueError: This matrix has no inverse.

    >>> inverse_of_matrix([[], []])
    Traceback (most recent call last):
    ...
    ValueError: Please provide a matrix of size 2x2 or 3x3.

    >>> inverse_of_matrix([[1, 2], [3, 4], [5, 6]])
    Traceback (most recent call last):
    ...
    ValueError: Please provide a matrix of size 2x2 or 3x3.

    >>> inverse_of_matrix([[1, 2, 1], [0, 3, 4]])
    Traceback (most recent call last):
    ...
    ValueError: Please provide a matrix of size 2x2 or 3x3.

    >>> inverse_of_matrix([[1, 2, 3], [7, 8, 9], [7, 8, 9]])
    Traceback (most recent call last):
```



```
...
ValueError: This matrix has no inverse.
]
>>> inverse_of_matrix([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
[[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 1.0]]
"""
]
d = Decimal
]
# Check if the provided matrix has 2 rows and 2 columns
# since this implementation only works for 2x2 matrices
if len(matrix) == 2 and len(matrix[0]) == 2 and len(matrix[1]) == 2:
# Calculate the determinant of the matrix
determinant = float(
d(matrix[0][0]) * d(matrix[1][1]) - d(matrix[1][0]) * d(matrix[0][1])
)
if determinant == 0:
raise ValueError("This matrix has no inverse.")
]
# Creates a copy of the matrix with swapped positions of the elements
swapped_matrix = [[0.0, 0.0], [0.0, 0.0]]
swapped_matrix[0][0], swapped_matrix[1][1] = matrix[1][1], matrix[0][0]
swapped_matrix[1][0], swapped_matrix[0][1] = -matrix[1][0], -matrix[0][1]
]
# Calculate the inverse of the matrix
return [
[(float(d(n)) / determinant) or 0.0 for n in row] for row in
swapped_matrix
]
elif (
len(matrix) == 3
and len(matrix[0]) == 3
and len(matrix[1]) == 3
and len(matrix[2]) == 3
):
# Calculate the determinant of the matrix using Sarrus rule
determinant = float(
(
(d(matrix[0][0]) * d(matrix[1][1]) * d(matrix[2][2]))
+ (d(matrix[0][1]) * d(matrix[1][2]) * d(matrix[2][0]))
+ (d(matrix[0][2]) * d(matrix[1][0]) * d(matrix[2][1]))
)
- (
(d(matrix[0][2]) * d(matrix[1][1]) * d(matrix[2][0]))
+ (d(matrix[0][1]) * d(matrix[1][0]) * d(matrix[2][2]))
+ (d(matrix[0][0]) * d(matrix[1][2]) * d(matrix[2][1]))
)
)
)
if determinant == 0:
raise ValueError("This matrix has no inverse.")
```



```
# Creating cofactor matrix
cofactor_matrix = [
[d(0.0), d(0.0), d(0.0)],
[d(0.0), d(0.0), d(0.0)],
[d(0.0), d(0.0), d(0.0)],
]

cofactor_matrix[0][0] = (d(matrix[1][1]) * d(matrix[2][2])) - (
d(matrix[1][2]) * d(matrix[2][1]))
)
cofactor_matrix[0][1] = -(
(d(matrix[1][0]) * d(matrix[2][2])) - (d(matrix[1][2]) * d(matrix[2][0]))
)
cofactor_matrix[0][2] = (d(matrix[1][0]) * d(matrix[2][1])) - (
d(matrix[1][1]) * d(matrix[2][0]))
)
cofactor_matrix[1][0] = -(
(d(matrix[0][1]) * d(matrix[2][2])) - (d(matrix[0][2]) * d(matrix[2][1]))
)
cofactor_matrix[1][1] = (d(matrix[0][0]) * d(matrix[2][2])) - (
d(matrix[0][2]) * d(matrix[2][0]))
)
cofactor_matrix[1][2] = -(
(d(matrix[0][0]) * d(matrix[2][1])) - (d(matrix[0][1]) * d(matrix[2][0]))
)
cofactor_matrix[2][0] = (d(matrix[0][1]) * d(matrix[1][2])) - (
d(matrix[0][2]) * d(matrix[1][1]))
)
cofactor_matrix[2][1] = -(
(d(matrix[0][0]) * d(matrix[1][2])) - (d(matrix[0][2]) * d(matrix[1][0]))
)
cofactor_matrix[2][2] = (d(matrix[0][0]) * d(matrix[1][1])) - (
d(matrix[0][1]) * d(matrix[1][0]))
)

# Transpose the cofactor matrix (Adjoint matrix)
adjoint_matrix = array(cofactor_matrix)
for i in range(3):
for j in range(3):
adjoint_matrix[i][j] = cofactor_matrix[j][i]

# Inverse of the matrix using the formula (1/determinant) * adjoint
matrix
inverse_matrix = array(adjoint_matrix) # FIX: usar adjoint_matrix, no
cofactor_matrix
for i in range(3):
for j in range(3):
inverse_matrix[i][j] /= d(determinant)

# Calculate the inverse of the matrix
```



```
return [[float(d(n)) or 0.0 for n in row] for row in inverse_matrix]  
raise ValueError("Please provide a matrix of size 2x2 or 3x3.")
```

Error lógico: En la inversión de la matriz 3x3, se usaba `cofactor_matrix` en lugar de `adjoint_matrix` al calcular la inversa, lo que producía resultados incorrectos en matrices no simétricas.

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱



- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".



3. Modificar la Vista (Template) de la Lista de Tareas:
 - Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
 - Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").
4. añadir Estilos (CSS) Opcionales:
 - Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

SOLUCION:

Pull Request hecho: <https://github.com/mjhea0/flaskr-tdd/pull/99>

Aquí está el código de los 3 archivos modificados:

models.py



```
python
from project.app import db

class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String, nullable=False)
    text = db.Column(db.String, nullable=False)
    priority = db.Column(
        db.String,
        nullable=False,
        default="Media"
    )

    def __init__(self, title, text, priority="Media"):
        self.title = title
        self.text = text
        self.priority = priority

    def __repr__(self):
        return f"<title {self.title}>"
```

app.py (solo la línea modificada)

```
python
new_entry = models.Post(request.form["title"], request.form["text"],
request.form.get("priority", "Media"))
```

index.html (las partes modificadas)

```
html
<!-- Selector de prioridad en el formulario -->
<dt>Prioridad:</dt>
<dd>
    <select name="priority" class="form-control col-sm-4">
        <option value="Alta">Alta</option>
        <option value="Media" selected>Media</option>
        <option value="Baja">Baja</option>
    </select>
</dd>
```



```
<!-- Etiqueta de prioridad en la lista -->
<span class="badge {% if entry.priority == 'Alta' %}bg-danger
  {% elif entry.priority == 'Media' %}bg-warning text-dark
  {% else %}bg-success{% endif %}">
  {{ entry.priority }}
</span>
```

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Viany Fernanda Feged Santos

Jefferson Yahir Cardenas Romero

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos

Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:



- Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
- Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

RESPUESTA: Documento: matrix/ inverse_of_matrix.py

1. Error lógico en el cálculo de la inversa de matrices 3x3

En la línea:

```
inverse_matrix = array (cofactor_matrix)
```

el programa utilizaba la matriz de cofactores para calcular la matriz inversa. Sin embargo, matemáticamente la fórmula correcta requiere utilizar la matriz adjunta, que corresponde a la transpuesta de la matriz de cofactores.

La fórmula correcta para calcular la inversa es:



$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$$

Por esta razón, se corrigió la línea por:

```
inverse_matrix = array(adjoint_matrix)
```

Con esta modificación el programa calcula correctamente la inversa de matrices 3x3.

2. Mejora para pruebas y visualización de resultados

Se agregó un bloque con `print()` para facilitar las pruebas del programa y visualizar los resultados en consola.

Código agregado:

```
if __name__ == "__main__":  
  
    matrix_2x2 = [[2, 5], [2, 0]]  
    print("Inverse 2x2:")  
    print(inverse_of_matrix(matrix_2x2))  
  
    matrix_3x3 = [[2, 5, 7], [2, 0, 1], [1, 2, 3]]  
    print("\nInverse 3x3:")  
    print(inverse_of_matrix(matrix_3x3))
```

Esto no corrige un error lógico ni de sintaxis, pero permite comprobar fácilmente el funcionamiento del programa.

Tipos de errores identificados

Error	Tipo de error
Uso incorrecto de cofactor_matrix en vez de adjoint_matrix	Error lógico
Falta de pruebas visuales del programa	Mejora de funcionalidad

Conclusión: Se identificó y corrigió un error lógico importante relacionado con el cálculo de la inversa de matrices 3x3. Además, se añadieron pruebas de ejecución para verificar el correcto funcionamiento del programa y facilitar la visualización de resultados.



Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"



- "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.



Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

RESPUESTA:

Link pull Request: <https://github.com/mjhea0/flaskr-tdd/pull/101#issue-4456620147>

Te adjunte la carpeta del código en zip del proyecto con las modificaciones profe para que lo compruebes en vs code.

PRUEBAS DE FUNCIONALIDAD:



The screenshot shows a VS Code editor with a project named 'FLASKR-TDD'. The file explorer on the left shows the project structure, including files like 'requirements.txt', 'models.py', 'app.py', and 'index.html'. The main editor window displays the 'models.py' file, which contains a SQLAlchemy model for a 'Post' class. The code defines columns for 'id', 'title', 'text', and 'priority'. The terminal at the bottom shows the output of running the application, including a warning about using a development server and the URL 'http://127.0.0.1:5000'.

```
1 from project.app import db
2
3
4 class Post(db.Model):
5     id = db.Column(db.Integer, primary_key=True)
6     title = db.Column(db.String, nullable=False)
7     text = db.Column(db.String, nullable=False)
8     priority = db.Column(db.String(10), default="Media")
9
10     def __init__(self, title, text, priority="Media"):
11         self.title = title
12         self.text = text
13         self.priority = priority
14
15     def __repr__(self):
16         return f"<title {self.title}>"
17
```

PROBLEMS: OUTPUT TERMINAL PORTS

```
127.0.0.1 - - [15/May/2026 15:03:34] "GET /static/main.js HTTP/1.1" 304 -
127.0.0.1 - - [15/May/2026 15:03:46] "POST /add HTTP/1.1" 302 -
127.0.0.1 - - [15/May/2026 15:03:46] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/May/2026 15:03:46] "GET /static/main.js HTTP/1.1" 304 -
PS C:\Users\valentina\flaskr-tdd> python -m flask run
* Serving Flask app 'project.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

The screenshot shows the Flask application running in a web browser at the URL '127.0.0.1:5000'. The page has a title 'Flaskr' and a navigation bar with 'Search' and 'log out' buttons. Below the navigation bar, there are three input fields: 'Title:', 'Text:', and 'Priority:'. The 'Priority:' field has a dropdown menu with options: 'Media', 'Alta', 'Media', and 'Baja'. The 'Media' option is selected. At the bottom of the page, there is a section titled 'tarea'.

Flaskr

Search log out

Title:

Text:

Priority:

Media

Alta

Media

Baja

tarea



127.0.0.1:5000

Comprueba que eres tú

Priority:

Baja

Share

- **tarea**
tarea
Prioridad: **Alta**
- **tarea1**
1
Prioridad: **Media**
- **h**
j
Prioridad: **Media**

Activar Windows
Ve a Configuración para activar Windows

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Frank Alejandro Morales

Kevin Andres Omaña Florez

Miguel Angel Solano Tarache

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos



Solución:

El error estaba en la parte donde se calculaba la matriz inversa de 3x3. Primero se construía correctamente la matriz adjunta (adjoint_matrix) haciendo la transposición de la matriz de cofactores pero el problema era que calculaba la transpuesta correctamente, pero luego ignoraba ese resultado y seguía trabajando con la matriz de cofactores original. El error se ubica en las siguientes líneas de código

```
# Transpose the cofactor matrix (Adjoint matrix)
adjoint_matrix = array(cofactor_matrix)
for i in range(3):
    for j in range(3):
        adjoint_matrix[i][j] = cofactor_matrix[j][i]

# Inverse of the matrix using the formula (1/determinant) * adjoint matrix
inverse_matrix = array(cofactor_matrix)
for i in range(3):
    for j in range(3):
        inverse_matrix[i][j] /= d(determinant)
```

También requería de la librería "NumPy" la cual se instala con:
"pip install numpy"

NumPy se estaba utilizando para manejar matrices de manera más eficiente mediante arreglos multidimensionales (array). Esto facilita operaciones matemáticas como divisiones entre todos los elementos de una matriz y transposiciones.

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀



- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"



- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🍷
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:



1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

<https://github.com/fmoralesz26/flaskr-tdd/pull/1>

App.py:

```
import os
```

```
from functools import wraps
```

```
from pathlib import Path
```

```
from flask import (
```

```
    Flask,
```

```
    render_template,
```

```
    request,
```

```
    session,
```

```
    flash,
```

```
    redirect,
```

```
    url_for,
```



```
abort,

jsonify,
)

from flask_socketio import SocketIO, send, emit

from flask_sqlalchemy import SQLAlchemy

basedir = Path(__file__).resolve().parent


# configuration

DATABASE = "flaskr.db"

USERNAME = "admin"

PASSWORD = "admin"

SECRET_KEY = "change_me"

url = os.getenv("DATABASE_URL", f"sqlite:///{"Path(basedir).joinpath(DATABASE)}")

if url.startswith("postgres://"):

    url = url.replace("postgres://", "postgresql://", 1)

SQLALCHEMY_DATABASE_URI = url

SQLALCHEMY_TRACK_MODIFICATIONS = False


# create and initialize a new Flask app

app = Flask(__name__)

# load the config
```



```
app.config.from_object(__name__)

# init sqlalchemy
db = SQLAlchemy(app)

import models

def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if not session.get("logged_in"):
            flash("Please log in.")
            return jsonify({"status": 0, "message": "Please log in."}), 401
        return f(*args, **kwargs)

    return decorated_function

@app.route("/")
def index():
    """Searches the database for entries, then displays them."""
    entries = db.session.query(models.Post)
    return render_template("index.html", entries=entries)

@app.route("/add", methods=["POST"])
def add_entry():
```




```
if not session.get("logged_in"):
```

```
    abort(401)
```

```
# 1. Extraer los tres campos enviados desde el HTML
```

```
title = request.form["title"]
```

```
text = request.form["text"]
```

```
priority = request.form.get("priority", "Media")
```

```
# 2. Pasar los tres argumentos al constructor del modelo
```

```
new_entry = models.Post(title, text, priority)
```

```
db.session.add(new_entry)
```

```
db.session.commit()
```

```
flash("New entry was successfully posted")
```

```
return redirect(url_for("index"))
```

```
@app.route("/login", methods=["GET", "POST"])
```

```
def login():
```

```
    """User login/authentication/session management."""
```

```
    error = None
```

```
    if request.method == "POST":
```

```
        if request.form["username"] != app.config["USERNAME"]:
```

```
            error = "Invalid username"
```

```
        elif request.form["password"] != app.config["PASSWORD"]:
```



```
        error = "Invalid password"

    else:

        session["logged_in"] = True

        flash("You were logged in")

        return redirect(url_for("index"))

    return render_template("login.html", error=error)


@app.route("/logout")
def logout():

    """User logout/authentication/session management."""

    session.pop("logged_in", None)

    flash("You were logged out")

    return redirect(url_for("index"))


@app.route("/delete/<int:post_id>", methods=["GET"])
@login_required
def delete_entry(post_id):

    """Deletes post from database."""

    result = {"status": 0, "message": "Error"}

    try:

        new_id = post_id

        db.session.query(models.Post).filter_by(id=new_id).delete()

        db.session.commit()

        result = {"status": 1, "message": "Post Deleted"}
```



```
flash("The entry was deleted.")

except Exception as e:

    result = {"status": 0, "message": repr(e)}

return redirect(url_for("index"))


@app.route("/search/", methods=["GET"])
def search():

    query = request.args.get("query")

    entries = db.session.query(models.Post)

    if query:

        return render_template("search.html", entries=entries, query=query)

    return render_template("search.html")


if __name__ == "__main__":

    app.run()
```

Models.py:

```
from app import db


class Post(db.Model):

    __tablename__ = "posts"


    id = db.Column(db.Integer, primary_key=True)

    title = db.Column(db.String(100), nullable=False)
```



```
text = db.Column(db.String(500), nullable=False)
```

```
priority = db.Column(db.String(10), default="Media")
```

```
def __init__(self, title, text, priority="Media"):
```

```
    self.title = title
```

```
    self.text = text
```

```
    self.priority = priority
```

```
def __repr__(self):
```

```
    return f"<Post {self.title}>"
```

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Andy Gabriel Rivera Calderon

Jesús David Arenales Cuervo

Greeicy Julieth Granados Jaimes

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "b_subbleort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos



Solución: Inverse of matrix

1. Error Lógico: Uso de matriz incorrecta en el cálculo final

-> Descripción: En el bloque de matrices 3x3, el código realiza el cálculo final dividiendo los elementos de la cofactor_matrix por el determinante. Sin embargo, para hallar la inversa, se debe utilizar la matriz adjunta (que es la traspuesta de la de cofactores). El código ya había calculado la **adjoint_matrix** pero no la estaba usando.

Tipo de error: Lógico.

Código con error:

```
# Error: Se vuelve a usar la matriz de cofactores en lugar de la adjunta
inverse_matrix = array(cofactor_matrix)
for i in range(3):
    for j in range(3):
        inverse_matrix[i][j] /= d(determinant)
```

Versión corregida:

Python

```
# Corrección: Se utiliza la adjoint_matrix para aplicar la fórmula de la inversa
inverse_matrix = array(adjoint_matrix)
for i in range(3):
    for j in range(3):
        inverse_matrix[i][j] /= d(determinant)
```

2. Error de Nombres: Uso de alias poco descriptivos

-> Descripción: Se define d = Decimal. Según la "Lista de Errores Comunes" y las convenciones de Python (PEP 8), las variables deben tener nombres descriptivos. El uso de una sola letra dificulta la lectura y el mantenimiento del código.

Tipo de error: Nombres / Convenciones.



Código con error:

```
d = Decimal
determinant = float(d(matrix[0][0]) * d(matrix[1][1]) ...)
```

Versión corregida:

```
# Se usa el nombre de la clase directamente para mayor claridad
determinant = float(Decimal(matrix[0][0]) * Decimal(matrix[1][1]) ...)
```

3. Error Lógico: Cálculo del determinante (Regla de Sarrus)

->Descripción: Al aplicar la Regla de Sarrus para la matriz 3x3, los índices de la diagonal secundaria estaban mal agrupados en los paréntesis, lo que daría un resultado matemático erróneo en ciertos casos.

Tipo de error: Lógico (Cálculo incorrecto).

Código con error:

```
- (
    (d(matrix[0][2]) * d(matrix[1][1]) * d(matrix[2][0]))
    + (d(matrix[0][1]) * d(matrix[1][0]) * d(matrix[2][2])) # Índices
incorrectos
    + (d(matrix[0][0]) * d(matrix[1][2]) * d(matrix[2][1]))
)
```

Versión corregida:

```
# Índices corregidos según la fórmula estándar de Sarrus
- (
    (d(matrix[2][0]) * d(matrix[1][1]) * d(matrix[0][2]))
    + (d(matrix[2][1]) * d(matrix[1][2]) * d(matrix[0][0]))
    + (d(matrix[2][2]) * d(matrix[1][0]) * d(matrix[0][1]))
)
```

4. Error de Sintaxis / Lógica: Redundancia en la transposición

->Descripción: El código importa numpy.array, pero luego realiza la transposición de la matriz de cofactores de forma manual con bucles for anidados. Esto es ineficiente y propenso a errores de índice cuando ya se cuenta con herramientas de librería.

Tipo de error: Lógica / Eficiencia.



Código con error:

```
adjoint_matrix = array(cofactor_matrix)
for i in range(3):
    for j in range(3):
        adjoint_matrix[i][j] = cofactor_matrix[j][i]
```

Versión corregida:

Se utiliza el método propio de numpy para transponer matrices

```
adjoint_matrix = array(cofactor_matrix).transpose()
```

5. Error de Sintaxis: Comparación incompleta en validación

-> Descripción: En la validación de la matriz 2x2, la condición if es excesivamente larga y manual. Aunque no rompe el código, aumenta el riesgo de errores de escritura (typos).

Tipo de error: Sintaxis / Estilo.

Código con error:

```
if len(matrix) == 2 and len(matrix[0]) == 2 and len(matrix[1]) == 2:
```

Versión corregida:

```
# Una forma más robusta y limpia de validar dimensiones
if len(matrix) == 2 and all(len(row) == 2 for row in matrix):
```

6. Error de Sintaxis y Dependencias (Importaciones)

Descripción: El código presentaba dos fallos críticos de ejecución: 1) La importación `from __future__ import annotations` no estaba en la primera línea del archivo, lo que genera un error de interpretación. 2) Se intentaba importar la librería externa `numpy`, la cual no está garantizada en todos los entornos de ejecución estándar, causando un `ModuleNotFoundError`.

Tipo de error: Sintaxis / Dependencia externa.

Código con error:

```
# (Había espacio o código antes de esta línea)
```



```
from __future__ import annotations
from numpy import array # Dependencia no instalada
```

Versión corregida:

```
from __future__ import annotations # Ahora como primera línea del archivo
# Se eliminó numpy y se reemplazó por lógica nativa de listas de Python
```



Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"



- "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:



1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)

Link Pull Request: <https://github.com/mihea0/flaskr-tdd/pull/100>

Códigos modificados (lo subrayado):



```
models.py X style.css
project > models.py > Post > __init__
1 from project.app import db
2
3
4 class Post(db.Model):
5     PRIORITY_CHOICES = ("Alta", "Media", "Baja")
6
7     id = db.Column(db.Integer, primary_key=True)
8     title = db.Column(db.String, nullable=False)
9     text = db.Column(db.String, nullable=False)
10    priority = db.Column(db.String, nullable=False, default="Media")
11
12    def __init__(self, title, text, priority="Media"):
13        self.title = title
14        self.text = text
15        self.priority = priority if priority in self.PRIORITY_CHOICES else "Media"
16
17    def __repr__(self):
18        return f"<title {self.title}>"
19
```

```
models.py app.py 1 X
project > app.py > add_entry
45 @login_required()
53 return decorated_function
54
55
56 @app.route("/")
57 def index():
58     """Searches the database for entries, then displays them."""
59     entries = db.session.query(models.Post)
60     return render_template("index.html", entries=entries)
61
62
63 @app.route("/add", methods=["POST"])
64 def add_entry():
65     """Adds new post to the database."""
66     if not session.get("logged_in"):
67         abort(401)
68     priority = request.form.get("priority", "Media")
69     new_entry = models.Post(request.form["title"], request.form["text"], priority)
70     db.session.add(new_entry)
71     db.session.commit()
72     flash("New entry was successfully posted")
73     return redirect(url_for("index"))
74
75
76 @app.route("/login", methods=["GET", "POST"])
77 def login():
78     """User login/authentication/session management."""
79     error = None
80     if request.method == "POST":
81         if request.form["username"] != app.config["USERNAME"]:
82             error = "Invalid username"
```



```
models.py index.html X
project > templates > index.html > html > head > link
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Flaskr</title>
5     <link
6       rel="stylesheet"
7       type="text/css"
8       href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
9     />
10    <link
11      rel="stylesheet"
12      type="text/css"
13      href="{{ url_for('static', filename='style.css') }}"
14    />
15  </head>
16  <body>
17    <div class="container">
18      <br /><br />
19      <h1>Flaskr</h1>
20      <br /><br />
21
22      <a class="btn btn-info" role="button" href="{{ url_for('search') }}">Search</a>
23
24      {% if not session.logged_in %}
25      <a class="btn btn-success" role="button" href="{{ url_for('login') }}"
26        >log in</a>
27      >
28      {% else %}
29      <a class="btn btn-warning" role="button" href="{{ url_for('logout') }}"
30        >log out</a>
31      </body>
32</html>
```

```
models.py index.html X
project > templates > index.html > html > body > div.container > form.add-entry.form-group > dl >
2 <html>
16 <body>
17   <div class="container">
41     <form>
46       <dl>
57         <dd>
58           <textarea
59             name="text"
60             rows="5"
61             cols="40"
62             class="form-control col-sm-4"
63           ></textarea>
64         </dd>
65         <dt>Prioridad:</dt>
66         <dd>
67           <select name="priority" class="form-control col-sm-4">
68             <option value="Alta">Alta</option>
69             <option value="Media" selected>Media</option>
70             <option value="Baja">Baja</option>
71           </select>
72         </dd>
73         <br /><br />
74         <dd>
75           <input type="submit" class="btn btn-primary" value="Share" />
76         </dd>
77       </dl>
78     </form>
79     {% endif %}
80   </div>
81 </body>
82</html>
```



```
models.py index.html X
project > templates > index.html > html > body > div.container > ul.entries > li.entry
2 <html>
16 <body>
17 <div class="container">
41 <form
46 <dl>
77 </dl>
78 </form>
79 {% endif %}
80
81 <br />
82
83 <ul class="entries">
84 <li class="entry">
85 <h2 id="{{ entry.id }}">
86 <span class="priority priority-{{ entry.priority|lower }}">
87 </span>
88 <span class="priority priority-{{ entry.priority|lower }}">
89 </span>
90 </h2>
91 <div class="text">
92 </div>
93 </li>
94 {% else %}
95 <li><em>No entries yet. Add some!</em></li>
96 {% endfor %}
97 </ul>
98 </div>
99 <script
100 type="text/javascript"
101 src="{{ url_for('static', filename='main.js') }}"
```

```
models.py search.html X
project > templates > search.html > html > head > link
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Flaskr</title>
5 <link
6 rel="stylesheet"
7 type="text/css"
8 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
9 />
10 <link
11 rel="stylesheet"
12 type="text/css"
13 href="{{ url_for('static', filename='style.css') }}"
14 />
15 </head>
16 <body>
17 <div class="container">
18 <br /><br />
19 <h1>Flaskr</h1>
20 <br /><br />
21
22 <a class="btn btn-primary" role="button" href="{{ url_for('index') }}">
23 Home
24 </a>
25
26 {% if not session.logged_in %}
27 <a class="btn btn-success" role="button" href="{{ url_for('login') }}">
28 log in</a>
29 >
30 {% else %}
31 <a class="btn btn-warning" role="button" href="{{ url_for('logout') }}">
32 log out</a>
33 >
```




```
models.py search.html X
project > templates > search.html > html > body > div.container > ul.entries > l.entry
2 <html>
16 <body>
17 <div class="container">
44 <form action="{{ url_for('search') }}" method="get" class="form-group">
45 <div>
51 <div><input type="submit" class="btn btn-info" value="Search" /></div>
52 </div>
53 </form>
54
55 <ul class="entries">
56 <li>{% for entry in entries %} {% if query.lower() in entry.title.lower() or
57 query.lower() in entry.text.lower() %}
58 <li class="entry">
59 <div><h2 id="{{ entry.post_id }}">
60 <div>{{ entry.title }}
61 <div><span class="priority priority-{{ entry.priority|lower }}">
62 <div>{{ entry.priority }}
63 </div></span>
64 </div></h2>
65 <div>{{ entry.text|safe }}
66 </div>
67 </li>
68 </ul>
69 </div>
70 <script
71 type="text/javascript"
72 src="{{ url_for('static', filename='main.js') }}"
73 ></script>
74 </body>
75 </html>
76
```

```
models.py style.css X
project > static > style.css > .priority-baja
71 .error {
72 background-color: #f0d6d6;
73 padding: 0.5em;
74 }
75
76 .priority {
77 display: inline-block;
78 font-size: 0.65em;
79 font-weight: bold;
80 padding: 0.2em 0.6em;
81 margin-left: 0.5em;
82 border-radius: 0.4em;
83 color: #fff;
84 vertical-align: middle;
85 text-transform: uppercase;
86 letter-spacing: 0.05em;
87 }
88
89 .priority-alta {
90 background-color: #d9534f;
91 }
92
93 .priority-media {
94 background-color: #f0ad4e;
95 color: #333;
96 }
97
98 .priority-baja {
99 background-color: #5cb85c;
100 }
101
```



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Cristhian David Maldonado Chiquillo

John Jayro Morales Montañez

Danna Tatiana Contreras Duarte

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "inverse_of_matrix.py" que se encuentra dentro de la carpeta "matrix" para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos



SOLUCIÓN ACTIVIDAD 1: Correcciones y Observaciones Encontradas

1. Validación de errores de sintaxis

Se revisó:

- Uso correcto de funciones
- Condicionales
- Bucles
- Importaciones
- Indentación
- Operadores matemáticos

Resultado:

No se encontraron errores de sintaxis.

2. Validación de tamaño de matriz

El código contiene una validación importante:

```
raise ValueError("Please provide a matrix of size 2x2 or 3x3.")
```

Explicación:

La función únicamente acepta matrices:

- 2x2
- 3x3

Si el usuario ingresa otro tamaño, el programa genera un error controlado.



Tipo:

Validación lógica.

3. Verificación del determinante

El programa valida si la matriz tiene inversa:

```
if determinant == 0:  
    raise ValueError("This matrix has no inverse.")
```

Explicación:

Si el determinante es cero, la matriz no puede invertirse.

Fórmula del determinante 2x2

$\det(A) = ad - bc$

Tipo:

Error matemático controlado.

4. Uso de Decimal para precisión

El código utiliza:

```
from decimal import Decimal
```

Explicación:

Esto ayuda a evitar errores de precisión con números decimales.

Beneficio:

Mayor exactitud en operaciones matemáticas.



5. Posible mejora encontrada

En la parte del cálculo de la matriz inversa 3x3:

```
inverse_matrix = array(cofactor_matrix)
```

Observación:

La variable debería inicializarse usando la matriz adjunta (adjoint_matrix) para seguir correctamente la fórmula matemática.

Corrección sugerida:

```
inverse_matrix = array(adjoint_matrix)
```

Tipo:

Posible mejora lógica/matemática.

Ejemplo de Funcionamiento

Matriz 2x2

```
inverse_of_matrix([[2, 5], [2, 0]])
```

Resultado:

```
[[0.0, 0.5], [0.2, -0.2]]
```

Complejidad del Algoritmo

Para matrices pequeñas (2x2 y 3x3), el tiempo de ejecución es constante:

$O(1)$

Conclusión



Durante la revisión del archivo `inverse_of_matrix.py` no se encontraron errores de sintaxis. El programa implementa correctamente el cálculo de matrices inversas para tamaños 2x2 y 3x3, utilizando validaciones para evitar operaciones inválidas. Además, se identificó una posible mejora lógica relacionada con el uso de la matriz adjunta durante el cálculo de la inversa en matrices 3x3.

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱



- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:



- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
 - Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").
4. añadir Estilos (CSS) Opcionales:
- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)



- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Nombres y apellidos completos:

Breekman Anderson Martinez Pérez

Juan Sebastián Flores Bernal

Yeison Estiven Meaury Moncada

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

SOLUCION

1. Tiene que instalar la biblioteca numpy la cual está diseñada para manipular de forma rápida y eficiente grandes volúmenes de datos numérico



2. En la línea 148 se vio el error de repetir el cofactor porque para hallar la inversa de una matriz se halla solamente se halla sus respectivos cofactores solo 1 vez y no dos veces y esos cofactores forman una matriz llamada matriz adjunta y con esa matriz adjunta se halla la inversa de una matriz

- `inverse_matrix = array(cofactor_matrix)`

siendo esta la solución cambiar el cofactor matriz por adjunto de una matriz

```
inverse_matrix = array(adjoint_matrix)
```

3. Y el error principal es que en el código no se utiliza print para poder mostrar los valores por ende el programa no podía ejecutarse.

```
• if __name__ == "__main__":  
•  
•     matrix_2x2 = [[2, 5], [2, 0]]  
•     print("Inverse 2x2:")  
•     print(inverse_of_matrix(matrix_2x2))  
•  
•     matrix_3x3 = [[2, 5, 7], [2, 0, 1], [1, 2, 3]]  
•     print("\nInverse 3x3:")  
•     print(inverse_of_matrix(matrix_3x3))
```

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2



- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️



Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado `priority` al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada `prioridad-tareas`.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").



5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Link.

<https://github.com/mjhea0/flaskr-tdd/pull/98>

Codigo:

project/models.py

```
from project.app import db

class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String, nullable=False)
    text = db.Column(db.String, nullable=False)
    priority = db.Column(db.String, default="Media", nullable=False)

    def __init__(self, title, text, priority=None):
        self.title = title
        self.text = text
        self.priority = priority

    def __repr__(self):
        return f"<title {self.title}, priority {self.priority}>"
```

templates/index.html

```
<dd>
  <select name="priority" class="form-select col-sm-4" style="max-width:
320px;">
    <option value="Baja">Baja</option>
    <option value="Media" selected>Media</option>
    <option value="Alta">Alta</option>
  </select>
```




```
</dd>
```

```
{% if entry.priority == 'Alta' %}
    <span class="badge bg-danger"> Alta prioridad</span>
    {% elif entry.priority == 'Media' %}
    <span class="badge bg-warning text-dark">Media
prioridad</span>
    {% elif entry.priority == 'Baja' %}
    <span class="badge bg-success">Baja prioridad</span>
{% endif %}
```

static/style.css

```
.badge {
    font-size: 0.85rem;
    padding: 6px 12px;
    border-radius: 20px;
    font-weight: 600;
    letter-spacing: 0.5px;
}

.priority-high {
    background-color: #dc3545 !important;
    box-shadow: 0 2px 6px rgba(220, 53, 69, 0.4);
    animation: pulse 1.5s infinite;
}

.priority-medium {
    background-color: #ffc107 !important;
    color: #212529 !important;
    box-shadow: 0 2px 6px rgba(255, 193, 7, 0.4);
}

.priority-low {
    background-color: #198754 !important;
    box-shadow: 0 2px 6px rgba(25, 135, 84, 0.4);
}

/* Animación para prioridad alta */
@keyframes pulse {
    0% { transform: scale(1); }
```



```
50% { transform: scale(1.07); }  
100% { transform: scale(1); }  
}
```

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

José Lisandro Mojica Cortes

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

R/ errores:

Error de Importación

Línea 5: si `numpy` no está instalado genera un `ImportError`.

```
from numpy import array
#correccion
pip install numpy
```



Error de Nombres

Línea 64: Variable con nombre poco descriptivo

```
d = Decimal
#correaccion
decimal_precision = Decimal
```

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀



Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:



- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
 - Establecer un valor por defecto para la prioridad (por ejemplo, "Media").
2. Modificar el Formulario de Creación/Edición de Tareas:
 - Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
 - Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".
 3. Modificar la Vista (Template) de la Lista de Tareas:
 - Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
 - Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").
 4. añadir Estilos (CSS) Opcionales:
 - Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.



R/ LINK de Pull Request: <https://github.com/mjhea0/flaskr-tdd/pull/105>

Códigos modificados: <https://github.com/jlmc03/flaskr-tdd/blob/prioridad-tareas/project/models.py>

<https://github.com/jlmc03/flaskr-tdd/blob/prioridad-tareas/project/app.py>

<https://github.com/jlmc03/flaskr-tdd/blob/prioridad-tareas/project/templates/index.html>

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

**Israel David Rodriguez Diaz
Eilyn Mayleth Villanueva Useche**

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

RTA:

Esto primero no es un error concreto, pero en la mayoría de IDEs no está instalada la librería numpy, haciendo que a muchos no les funcione este código, solución, instalarla, en muchos casos puede ser un poco complicado, así que se podría requerir ayuda de alguna inteligencia artificial o algún video en internet.



Otro problema con este código es que, no tiene pies ni cabeza, no hay inputs donde uno pueda ingresar números para una matriz, y aun no hay siquiera un print para mostrarnos el resultado, en un código sin capacidad de ejecución, en el siguiente tramo procuraré integrarle inputs y prints de manera de que se pueda usar de manera práctica.

```
from __future__ import annotations

from decimal import Decimal
from numpy import array

def inverse_of_matrix(matrix: list[list[float]]) -> list[list[float]]:

    d = Decimal

    if len(matrix) == 2 and len(matrix[0]) == 2 and len(matrix[1]) == 2:

        determinant = float(
            d(matrix[0][0]) * d(matrix[1][1]) - d(matrix[1][0]) *
d(matrix[0][1])
        )

        print(f"\nDeterminante 2x2: {determinant}")

        if determinant == 0:
            raise ValueError("This matrix has no inverse.")

        swapped_matrix = [[0.0, 0.0], [0.0, 0.0]]

        swapped_matrix[0][0], swapped_matrix[1][1] = matrix[1][1],
matrix[0][0]
        swapped_matrix[1][0], swapped_matrix[0][1] = -matrix[1][0], -
matrix[0][1]

        print("\nMatriz intercambiada:")
```



```
print(swapped_matrix)

inverse_result = [
    [(float(d(n)) / determinant) or 0.0 for n in row]
    for row in swapped_matrix
]

return inverse_result

elif (
    len(matrix) == 3
    and len(matrix[0]) == 3
    and len(matrix[1]) == 3
    and len(matrix[2]) == 3
):

    determinant = float(
        (
            (d(matrix[0][0]) * d(matrix[1][1]) * d(matrix[2][2]))
            + (d(matrix[0][1]) * d(matrix[1][2]) * d(matrix[2][0]))
            + (d(matrix[0][2]) * d(matrix[1][0]) * d(matrix[2][1]))
        )
        - (
            (d(matrix[0][2]) * d(matrix[1][1]) * d(matrix[2][0]))
            + (d(matrix[0][1]) * d(matrix[1][0]) * d(matrix[2][2]))
            + (d(matrix[0][0]) * d(matrix[1][2]) * d(matrix[2][1]))
        )
    )

    print(f"\nDeterminante 3x3: {determinant}")

    if determinant == 0:
        raise ValueError("This matrix has no inverse.")

    cofactor_matrix = [
        [d(0.0), d(0.0), d(0.0)],
```



```
[d(0.0), d(0.0), d(0.0)],  
[d(0.0), d(0.0), d(0.0)],  
]  
  
cofactor_matrix[0][0] = (d(matrix[1][1]) * d(matrix[2][2])) - (  
    d(matrix[1][2]) * d(matrix[2][1])  
)  
  
cofactor_matrix[0][1] = -(  
    (d(matrix[1][0]) * d(matrix[2][2]))  
    - (d(matrix[1][2]) * d(matrix[2][0])))  
)  
  
cofactor_matrix[0][2] = (d(matrix[1][0]) * d(matrix[2][1])) - (  
    d(matrix[1][1]) * d(matrix[2][0])  
)  
  
cofactor_matrix[1][0] = -(  
    (d(matrix[0][1]) * d(matrix[2][2]))  
    - (d(matrix[0][2]) * d(matrix[2][1])))  
)  
  
cofactor_matrix[1][1] = (d(matrix[0][0]) * d(matrix[2][2])) - (  
    d(matrix[0][2]) * d(matrix[2][0])  
)  
  
cofactor_matrix[1][2] = -(  
    (d(matrix[0][0]) * d(matrix[2][1]))  
    - (d(matrix[0][1]) * d(matrix[2][0])))  
)  
  
cofactor_matrix[2][0] = (d(matrix[0][1]) * d(matrix[1][2])) - (  
    d(matrix[0][2]) * d(matrix[1][1])  
)  
  
cofactor_matrix[2][1] = -(
```



```
(d(matrix[0][0]) * d(matrix[1][2]))
- (d(matrix[0][2]) * d(matrix[1][0]))
)

cofactor_matrix[2][2] = (d(matrix[0][0]) * d(matrix[1][1])) - (
    d(matrix[0][1]) * d(matrix[1][0])
)

print("\nMatriz de cofactores:")
print(cofactor_matrix)

adjoint_matrix = array(cofactor_matrix)

for i in range(3):
    for j in range(3):
        adjoint_matrix[i][j] = cofactor_matrix[j][i]

print("\nMatriz adjunta:")
print(adjoint_matrix)

inverse_matrix = array(adjoint_matrix)

for i in range(3):
    for j in range(3):
        inverse_matrix[i][j] /= d(determinant)

return [[float(d(n)) or 0.0 for n in row] for row in inverse_matrix]

raise ValueError("Please provide a matrix of size 2x2 or 3x3.")

print("CALCULO DE MATRIZ INVERSA")

tam = int(input("Ingrese tamaño de la matriz (2 o 3): "))

matrix = []
```



```
for i in range(tam):

    fila = input(
        f"Ingrese los valores de la fila {i+1} separados por espacio: "
    ).split()

    matrix.append([float(x) for x in fila])

print("\nMatriz ingresada:")
print(matrix)

try:

    resultado = inverse_of_matrix(matrix)

    print("\nMatriz inversa:")

    for fila in resultado:
        print(fila)

except ValueError as e:

    print("\nERROR:")
    print(e)
```

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la



integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.

RTA

Las nuevas funcionalidades ya están implementadas en el código presentado en la actividad 1, la implementación permite ingresar y recibir datos.

- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷



- Un "commit" es como guardar una versión del archivo con una nota.
Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:
 - Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
 - Establecer un valor por defecto para la prioridad (por ejemplo, "Media").
2. Modificar el Formulario de Creación/Edición de Tareas:
 - Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
 - Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".
3. Modificar la Vista (Template) de la Lista de Tareas:
 - Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
 - Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").



4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Links de los pulls:

[Update models.py by israelrodriguezk13-sys · Pull Request #102 · mjhea0/flaskr-tdd](#)

[Update index.html by israelrodriguezk13-sys · Pull Request #103 · mjhea0/flaskr-tdd](#)

[Update style.css by israelrodriguezk13-sys · Pull Request #104 · mjhea0/flaskr-tdd](#)

Codigo:

project/models.py

```
from project.app import db

class Post(db.Model):
    __tablename__ = "post"

    post_id = db.Column(db.Integer, primary_key=True)
```



```
post_title = db.Column(db.String(150), nullable=False)
post_text = db.Column(db.String(500), nullable=False)
post_priority = db.Column(
    db.String(20),
    nullable=False,
    default="Media"
)

def __init__(self, title, text, priority="Media"):
    self.post_title = title
    self.post_text = text
    self.post_priority = priority
```

templates/index.html

```
<dd>
  <select name="priority" class="form-select col-sm-4" style="max-width:
320px;">
    <option value="Baja">Baja</option>
    <option value="Media" selected>Media</option>
    <option value="Alta">Alta</option>
  </select>
</dd>
```

```
{% if entry.priority == 'Alta' %}
    <span class="badge bg-danger"> Prioridad Alta</span>
{% elif entry.priority == 'Media' %}
    <span class="badge bg-warning text-dark">Prioridad
Media</span>
{% elif entry.priority == 'Baja' %}
    <span class="badge bg-success">Prioridad Baja</span>
{% endif %}
```

static/style.css

```
.badge {
  font-size: 0.9rem;
  padding: 7px 14px;
  border-radius: 18px;
```



```
font-weight: 700;
letter-spacing: 0.4px;
transition: all 0.25s ease;
}

/* Prioridad alta */
.priority-high {
  background: linear-gradient(135deg, #dc3545, #b02a37) !important;
  color: #fff;
  box-shadow: 0 3px 8px rgba(220, 53, 69, 0.35);
  animation: pulse 1.8s infinite;
}

/* Prioridad media */
.priority-medium {
  background: linear-gradient(135deg, #ffc107, #e0a800) !important;
  color: #212529 !important;
  box-shadow: 0 3px 8px rgba(255, 193, 7, 0.35);
}

/* Prioridad baja */
.priority-low {
  background: linear-gradient(135deg, #198754, #146c43) !important;
  color: #fff;
  box-shadow: 0 3px 8px rgba(25, 135, 84, 0.35);
}

/* Animación suave para prioridad alta */
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}
```

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)



- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Nombres y apellidos completos: Johan Fernando Saenz Sanguino

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos

Actividad 1: Caza de Errores (60 minutos) 🕵️



- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "inverse_of_matrix.py" que se encuentra dentro de la carpeta "matrix" para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos



SOLUCIÓN ACTIVIDAD 1: Correcciones y Observaciones Encontradas

1. Validación de errores de sintaxis

Se revisó:

- Uso correcto de funciones
- Condicionales
- Bucles
- Importaciones
- Indentación
- Operadores matemáticos

Resultado:

No se encontraron errores de sintaxis.

2. Validación de tamaño de matriz

El código contiene una validación importante:

```
raise ValueError("Please provide a matrix of size 2x2 or 3x3.")
```

Explicación:

La función únicamente acepta matrices:

- 2x2
- 3x3

Si el usuario ingresa otro tamaño, el programa genera un error controlado.

Tipo:

Validación lógica.



3. Verificación del determinante

El programa valida si la matriz tiene inversa:

```
if determinant == 0:  
    raise ValueError("This matrix has no inverse.")
```

Explicación:

Si el determinante es cero, la matriz no puede invertirse.

Fórmula del determinante 2x2

$\det(A) = ad - bc$

Tipo:

Error matemático controlado.

4. Uso de Decimal para precisión

El código utiliza:

```
from decimal import Decimal
```

Explicación:

Esto ayuda a evitar errores de precisión con números decimales.

Beneficio:

Mayor exactitud en operaciones matemáticas.

5. Posible mejora encontrada

En la parte del cálculo de la matriz inversa 3x3:



```
inverse_matrix = array(cofactor_matrix)
```

Observación:

La variable debería inicializarse usando la matriz adjunta (adjoint_matrix) para seguir correctamente la fórmula matemática.

Corrección sugerida:

```
inverse_matrix = array(adjoint_matrix)
```

Tipo:

Posible mejora lógica/matemática.

Ejemplo de Funcionamiento**Matriz 2x2**

```
inverse_of_matrix([[2, 5], [2, 0]])
```

Resultado:

```
[[0.0, 0.5], [0.2, -0.2]]
```

Complejidad del Algoritmo

Para matrices pequeñas (2x2 y 3x3), el tiempo de ejecución es constante:

$O(1)$

Conclusión

Durante la revisión del archivo inverse_of_matrix.py no se encontraron errores de sintaxis. El programa implementa correctamente el cálculo de matrices inversas para tamaños 2x2 y 3x3, utilizando validaciones para evitar operaciones inválidas. Además, se



identificó una posible mejora lógica relacionada con el uso de la matriz adjunta durante el cálculo de la inversa en matrices 3x3.

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷



- Un "commit" es como guardar una versión del archivo con una nota.
Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").



4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

**Israel David Rodriguez Diaz
Eilyn Mayleth Villanueva Useche**

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "bubble_sort.py" que se encuentra dentro de la carpeta "sorts" (ordenamientos) para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos

RTA:

Esto primero no es un error concreto, pero en la mayoría de IDEs no está instalada la librería numpy, haciendo que a muchos no les funcione este código, solución, instalarla, en muchos casos puede ser un poco complicado, así que se podría requerir ayuda de alguna inteligencia artificial o algún video en internet.



Otro problema con este código es que, no tiene pies ni cabeza, no hay inputs donde uno pueda ingresar números para una matriz, y aun no hay siquiera un print para mostrarnos el resultado, en un código sin capacidad de ejecución, en el siguiente tramo procuraré integrarle inputs y prints de manera de que se pueda usar de manera práctica.

```
from __future__ import annotations

from decimal import Decimal
from numpy import array

def inverse_of_matrix(matrix: list[list[float]]) -> list[list[float]]:

    d = Decimal

    if len(matrix) == 2 and len(matrix[0]) == 2 and len(matrix[1]) == 2:

        determinant = float(
            d(matrix[0][0]) * d(matrix[1][1]) - d(matrix[1][0]) *
d(matrix[0][1])
        )

        print(f"\nDeterminante 2x2: {determinant}")

        if determinant == 0:
            raise ValueError("This matrix has no inverse.")

        swapped_matrix = [[0.0, 0.0], [0.0, 0.0]]

        swapped_matrix[0][0], swapped_matrix[1][1] = matrix[1][1],
matrix[0][0]
        swapped_matrix[1][0], swapped_matrix[0][1] = -matrix[1][0], -
matrix[0][1]

        print("\nMatriz intercambiada:")
```




```
print(swapped_matrix)

inverse_result = [
    [(float(d(n)) / determinant) or 0.0 for n in row]
    for row in swapped_matrix
]

return inverse_result

elif (
    len(matrix) == 3
    and len(matrix[0]) == 3
    and len(matrix[1]) == 3
    and len(matrix[2]) == 3
):

    determinant = float(
        (
            (d(matrix[0][0]) * d(matrix[1][1]) * d(matrix[2][2]))
            + (d(matrix[0][1]) * d(matrix[1][2]) * d(matrix[2][0]))
            + (d(matrix[0][2]) * d(matrix[1][0]) * d(matrix[2][1]))
        )
        - (
            (d(matrix[0][2]) * d(matrix[1][1]) * d(matrix[2][0]))
            + (d(matrix[0][1]) * d(matrix[1][0]) * d(matrix[2][2]))
            + (d(matrix[0][0]) * d(matrix[1][2]) * d(matrix[2][1]))
        )
    )

    print(f"\nDeterminante 3x3: {determinant}")

    if determinant == 0:
        raise ValueError("This matrix has no inverse.")

    cofactor_matrix = [
        [d(0.0), d(0.0), d(0.0)],
```



```
[d(0.0), d(0.0), d(0.0)],  
[d(0.0), d(0.0), d(0.0)],  
]  
  
cofactor_matrix[0][0] = (d(matrix[1][1]) * d(matrix[2][2])) - (  
    d(matrix[1][2]) * d(matrix[2][1])  
)  
  
cofactor_matrix[0][1] = -(  
    (d(matrix[1][0]) * d(matrix[2][2]))  
    - (d(matrix[1][2]) * d(matrix[2][0]))  
)  
  
cofactor_matrix[0][2] = (d(matrix[1][0]) * d(matrix[2][1])) - (  
    d(matrix[1][1]) * d(matrix[2][0])  
)  
  
cofactor_matrix[1][0] = -(  
    (d(matrix[0][1]) * d(matrix[2][2]))  
    - (d(matrix[0][2]) * d(matrix[2][1]))  
)  
  
cofactor_matrix[1][1] = (d(matrix[0][0]) * d(matrix[2][2])) - (  
    d(matrix[0][2]) * d(matrix[2][0])  
)  
  
cofactor_matrix[1][2] = -(  
    (d(matrix[0][0]) * d(matrix[2][1]))  
    - (d(matrix[0][1]) * d(matrix[2][0]))  
)  
  
cofactor_matrix[2][0] = (d(matrix[0][1]) * d(matrix[1][2])) - (  
    d(matrix[0][2]) * d(matrix[1][1])  
)  
  
cofactor_matrix[2][1] = -(  

```



```
(d(matrix[0][0]) * d(matrix[1][2]))
- (d(matrix[0][2]) * d(matrix[1][0]))
)

cofactor_matrix[2][2] = (d(matrix[0][0]) * d(matrix[1][1])) - (
    d(matrix[0][1]) * d(matrix[1][0])
)

print("\nMatriz de cofactores:")
print(cofactor_matrix)

adjoint_matrix = array(cofactor_matrix)

for i in range(3):
    for j in range(3):
        adjoint_matrix[i][j] = cofactor_matrix[j][i]

print("\nMatriz adjunta:")
print(adjoint_matrix)

inverse_matrix = array(adjoint_matrix)

for i in range(3):
    for j in range(3):
        inverse_matrix[i][j] /= d(determinant)

return [[float(d(n)) or 0.0 for n in row] for row in inverse_matrix]

raise ValueError("Please provide a matrix of size 2x2 or 3x3.")

print("CALCULO DE MATRIZ INVERSA")

tam = int(input("Ingrese tamaño de la matriz (2 o 3): "))

matrix = []
```



```
for i in range(tam):

    fila = input(
        f"Ingrese los valores de la fila {i+1} separados por espacio: "
    ).split()

    matrix.append([float(x) for x in fila])

print("\nMatriz ingresada:")
print(matrix)

try:

    resultado = inverse_of_matrix(matrix)

    print("\nMatriz inversa:")

    for fila in resultado:
        print(fila)

except ValueError as e:

    print("\nERROR:")
    print(e)
```

Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la



integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.

RTA

Las nuevas funcionalidades ya están implementadas en el código presentado en la actividad 1, la implementación permite ingresar y recibir datos.

- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📸



- Un "commit" es como guardar una versión del archivo con una nota.
Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"
 - "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:
 - Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
 - Establecer un valor por defecto para la prioridad (por ejemplo, "Media").
2. Modificar el Formulario de Creación/Edición de Tareas:
 - Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
 - Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".
3. Modificar la Vista (Template) de la Lista de Tareas:
 - Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
 - Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").



4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.

Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Links de los pulls:

[Update models.py by israelrodriguezk13-sys · Pull Request #102 · mjhea0/flaskr-tdd](#)

[Update index.html by israelrodriguezk13-sys · Pull Request #103 · mjhea0/flaskr-tdd](#)

[Update style.css by israelrodriguezk13-sys · Pull Request #104 · mjhea0/flaskr-tdd](#)

Codigo:

project/models.py

```
from project.app import db

class Post(db.Model):
    __tablename__ = "post"

    post_id = db.Column(db.Integer, primary_key=True)
```



```
post_title = db.Column(db.String(150), nullable=False)
post_text = db.Column(db.String(500), nullable=False)
post_priority = db.Column(
    db.String(20),
    nullable=False,
    default="Media"
)

def __init__(self, title, text, priority="Media"):
    self.post_title = title
    self.post_text = text
    self.post_priority = priority
```

templates/index.html

```
<dd>
    <select name="priority" class="form-select col-sm-4" style="max-width:
320px;">
        <option value="Baja">Baja</option>
        <option value="Media" selected>Media</option>
        <option value="Alta">Alta</option>
    </select>
</dd>
```

```
{% if entry.priority == 'Alta' %}
    <span class="badge bg-danger"> Prioridad Alta</span>
{% elif entry.priority == 'Media' %}
    <span class="badge bg-warning text-dark">Prioridad
Media</span>
{% elif entry.priority == 'Baja' %}
    <span class="badge bg-success">Prioridad Baja</span>
{% endif %}
```

static/style.css

```
.badge {
    font-size: 0.9rem;
    padding: 7px 14px;
    border-radius: 18px;
```




```
font-weight: 700;
letter-spacing: 0.4px;
transition: all 0.25s ease;
}

/* Prioridad alta */
.priority-high {
  background: linear-gradient(135deg, #dc3545, #b02a37) !important;
  color: #fff;
  box-shadow: 0 3px 8px rgba(220, 53, 69, 0.35);
  animation: pulse 1.8s infinite;
}

/* Prioridad media */
.priority-medium {
  background: linear-gradient(135deg, #ffc107, #e0a800) !important;
  color: #212529 !important;
  box-shadow: 0 3px 8px rgba(255, 193, 7, 0.35);
}

/* Prioridad baja */
.priority-low {
  background: linear-gradient(135deg, #198754, #146c43) !important;
  color: #fff;
  box-shadow: 0 3px 8px rgba(25, 135, 84, 0.35);
}

/* Animación suave para prioridad alta */
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}
```

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)



- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)